The Dice Game

Project Report | Java Programming, IT-Högskolan | Single-player and multiplayer console game

1. Introduction

The Dice Game is a text-based Java console application with two game modes: a **single-player guessing game** and a **multiplayer highest-score game**. The code is organised into five classes in the package `Game`: **Main**, **DiceGame**, **MultiPlayerGame**, **Player** and **Die**.

2. Game Rules

- **Single player:** choose the number of rounds (> 0) and the number of die sides (≥ 2). Each round, guess a number between 1 and the number of sides; the die is rolled and a **correct guess earns +1 point**. The final score is shown at the end.
- **Multiplayer:** choose the number of players (≥ 2), dice per player (≥ 1, all six-sided) and rounds (> 0). Each round every player rolls all their dice and the **sum is added to their total**. The player with the **highest total wins**.

3. How to Start the Game

Requires a Java JDK. From the project root folder (the folder containing `Game/`) run:

```
javac Game/*.java
java Game.Main
```

Or run `Main.java` from an IDE. A menu appears: **1** = Single player, **2** = Multiplayer, **3** = Exit. After a game ends the menu is shown again.

4. Overview of Classes and Methods

Class	Method(s)	Used by	Purpose
Main	`main(String[] args)`	Both	Shows the game-mode menu in a loop, reads the choice as a String (so invalid input needs no try/catch), starts the selected mode and closes the shared Scanner on exit.
DiceGame / MultiPlayerGame	`startGame(Scanner)`	Both	Entry point of each mode. Both validate the number of rounds, play all rounds and print a GAME OVER summary. Single player also asks for die sides and the name; multiplayer calls initialize(), pauses between rounds and prints all scores and the winner.
DiceGame	`getValidGuess(Scanner, int)`	Single	Repeats until the guess is a whole number from 1 to sides.
	`playRounds(Player, Scanner, int, int)`	Single	Game loop: guess, roll, compare, award +1 point on a match, show the score.
MultiPlayerGame	`initialize(Scanner)`	Multi	Validates number of players and dice, reads non-blank names, creates each Player with six-sided dice.
	`takeTurn(ArrayList<Player>)`	Multi	Rolls every player's dice, sums the round, adds it to the total and prints a formatted table row.
	`getWinners(ArrayList<Player>)`	Multi	Finds the player with the highest total score.
Player	`Player(String)`	Both	Stores the name, starts the score at 0 and creates an empty list of dice.
	`getPlayerName(), getScore(), setScore(int), increaseScore()`	Both	Read and update the name and score. increaseScore() adds 1 (single player); setScore() adds the round total (multiplayer).
	`addDie(int), rollDice(), getDice()`	Both	Add a die with the given sides, roll all of the player's dice, and return the dice list.
	`getDieValue()`	Single	Returns the value of the first die.
Die	`Die(int), roll()`	Both	Stores the number of sides; roll() sets a random value with random.nextInt(sides) + 1.
	`getCurrentValue(), getNumberOfSides()`	Both	Return the last rolled value and the number of sides.

5. Key Coding Decisions

- **Separation of responsibilities:** Main handles the menu, DiceGame and MultiPlayerGame handle each mode's flow, Player holds name, score and dice, and Die only rolls. Both modes reuse the same Player and Die classes.
- **Encapsulation:** all fields are private and accessed through methods; the dice list in Player is final and created in the constructor.
- **Input validation:** every numeric input uses a while-loop with try/catch on InputMismatchException, so wrong input never crashes the game. Names in multiplayer cannot be blank, and the menu reads text to avoid exceptions.
- **Scanner handling:** one Scanner is created in Main and passed to each mode; scanner.nextLine() clears leftover input after nextInt().
- **Flexible dice:** Player stores an ArrayList<Die>, so the same class supports one die (single player) or many (multiplayer).
- **Readable code:** the logic is split into small private static helper methods, and printf is used to show the multiplayer results in aligned columns.